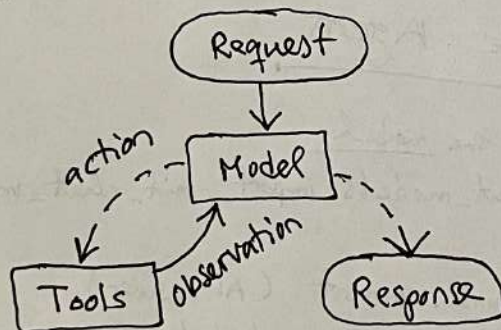


②

- Agents around a model
from langchain, agents impart `create_agent`
`agent = create_agent(model=model)`
 - create
 - invoke $\rightarrow$ `response['messages'][-1].content` (final AI answer)
- Streaming improves perceived UX/latency
 $\rightarrow$ stream tokens in real-time as they become available
- System prompt changes model's behavior and tone
 - Give role and task
 - Give optional examples (few-shot)
 - Give optional structure for output (in the prompt)
 $\rightarrow$ BAD practice
- Response format allows to define structured model output schema (Pydantic model)
 $\rightarrow$ `response["structured_response"].field1`
`.field2`
- Tools separate an agent from a standard chatbot
 $\rightarrow$ ability to take actions autonomously



ReAct agent

- Reason
- Act
- observe

- Coordinates a complex system that utilizes tools
- from langchain, tools import tool

from langchain, tool is imp.
 @tool ← name & description can be overrides here
 def square_root(x: float) → float:
 """Returns the square root of a number"""

name → description (detailed!) →

```
@tool ← name: square-root
def square-root(x: float) → float:
    """Calculate the square root of a number"""
    return x ** 0.5
```

→ invoke the tool directly (optional)
→ invoke the tool by giving it to an agent

- invoke the tool directly (optional)
- invoke the tool by giving it to an agent (system prompt may reference the available tools)
- When an agent invokes a tool, it creates an AIMessage w/ function call details and ToolMessage w/ function call response + final AIMessage response

③

- Web search tool using Tavily API

```
from langchain.tools import tool
from tavily import TavilyClient
tavily_client = TavilyClient()
```

@tool

```
def web_search(query: str) → Dict[str, Any]:
```

```
    """Search the web for information"""
```

```
    return tavily_client.search(query)
```

→ invoke the tool directly or indirectly by giving it to an agent

→ returns an object with query, follow-up questions, answer, images, results (array)

- url
- title
- content
- score
- raw_content

Agent can parse the results and respond with a human-readable content (final answer)

- LangSmith allows to debug, observe, and see details of a trace
- By default, agents do not have ~~any~~ memory between subsequent invocations

```
from langgraph, checkpoint, memory import InMemorySaver
agent = create_agent("gpt-5", checkpoint=InMemorySaver())
```

```
config = {"configurable": {"thread_id": "1"}}
```

```
response = agent.invoke({"messages": [question]}, config)
```

- By default, memory tracks messages only
- can be extended to cover custom fields like user ID, language

- Multimodal models can take images, audio, video as input and output

- Encoding: text → binary (base 2) → base 64 string

- image upload file → bytes → base 64

```
HumanMessage(content=[
```

```
    {"type": "text", "text": "Tell me about this capital"},
```

```
    {"type": "image", "base64": "img_base64", "mime_type": "image/png"}])
```

④

• audio ~~record~~ file → bytes → base64

agent = create_agent(model="gpt-4o-audio-preview")

HumanMessage(content=[

{ "type": "text", "text": "Tell me about this audio file?" },

{ "type": "audio", "base64": ~~audio_base64~~ audio_base64,

"mime_type": "audio/wav" }])

• Run Python file in LangSmith Studio UI

@ <https://smith.langchain.com/studio/?baseUrl=https://127.0.0.1:2024>

— Define langgraph.json

• "dependencies"

• "graphs" > "agent"

• "env"

→ uv run langgraph dev

• Project: Personal Chef

— web search tool powered by Tavily

— system prompt (role, task, tools, return)

— memory (not needed in the example, though)

Module 2: Advanced Agent

• Model Context Protocol (MCP): An open protocol that standardizes how your LLM apps connect to and work with your tools and data sources

— Prompts

— Tools

— Resources

MCP Host = your app

MCP Client

MCP Server

• Lots of open-source MCP servers with useful tools

• tools
• prompts
• resources

• Transport

1) stdio

(local MCP server)

2) streamable HTTP

(online MCP server)

• MCP server is like a web service, but for models

⑤

- stdio : runs a local MCP server (app) using a command + args

```
client = MultiServerMCPClient({
    "local_server": {
        "transport": "stdio",
        "command": "python",
        "args": ["resources/2.1-mcp-server.py"]
    }
})
```

```
tools = await client.get_tools()
```

```
resources = await client.get_resources("local_server")
prompts = await client.get_prompts("local_server")
prompt = prompts[0].content
```

- MCP server definition

```
mcp = FastMCP("mcp-server")
@mcp.tool()
def ...
@mcp.resource("githubs://...")
def ...
@mcp.prompt()
def ...
```

```
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

- streamable HTTP : runs on ~~the server~~ online MCP server (remote) using a URL

```
client = MultiServerMCPClient({
    "travel_server": {
        "transport": "streamable-http",
        "url": "https://mcp.kimi.com"
    }
})
```

- Context schema can be defined and passed to the agent
- Context is passed to tool calls in an object called tool runtime
- Only tool calls can access context

```
@tool
def get_favorite_color(runtime: ToolRuntime) -> str:
    """Get the favorite color of the user"""
    return runtime.context.favorite_color
```

⑥

```
agent = create_agent(model="gpt-5-nano",
                    tools=[get_favorite_color],
                    context_schema=ColourContext)
```

@dataclass

class ColourContext:

favorite_color: str = "blue"

```
agent.invoke({"messages": [...]} ,
            "context" = ColourContext())
```

↳ default value

• Context is like prompt templates

(but for tools!)
→ (variables = variables)

favorite_color = "green"
(override in runtime)

~~→ useful for defining agent's state~~

• from langchain.agents import AgentState
class CustomState(AgentState):

favorite_color: str

• agents can read and write state via tools

```
→ agent = create_agent("gpt-5-nano",
                        tools=[...],
                        checkpoint=InMemorySaver(),
                        state_schema=CustomState)
```

• agent.invoke(...) sync

• await agent.ainvoke(...) async

• state is agent's state. By default contains only messages

```
agent = create_agent("gpt-5-nano",
                    tools=[...],
                    checkpoint=InMemorySaver(),
                    state_schema=CustomState)
```

```
agent.invoke({"messages": [HumanMessage(content="My favorite color is green,")]} ,
            {"configurable": {"thread-id": "1"}})
```

- OR -

```
agent.invoke({"messages": [HumanMessage(content="Hello how are you?")],
            "favorite_color": "green" ,
            {"configurable": {"thread-id": "10"}}})
```

→ stored into the agent's state

⑦

- Note: context and state can be accessed (read & write) only via tool calls!

- Task complexity (single agent)

- too many tools

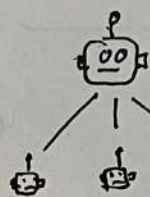
- overflowed context window

→ reduced output quality

- Multi-agent system = multiple specialized agents that work together to solve the problem

- add agents

as tools for the supervisor agent to call



Supervisor

- orchestration
- delegation

Sub agents

- research
- write
- edit

(via tools)
examples

```
subagent_1 = create_agent(
    model = "gpt-5-nano",
    tools = [tool_1])
```

```
subagent_2 = create_agent(
    model = "gpt-5-nano",
    tools = [tool_2])
```

```
main_agent = create_agent(
    model = "gpt-5-nano",
    tools = [call_subagent_1, call_subagent_2],
    system_prompt = "role, task, subagents (tools)")
```

@tool

```
def call_subagent_1(x: float) -> float:
```

""" calls subagent 1 in order to calculate the square root of a number. """

```
response = subagent_1.invoke({"messages": [
    HumanMessage(content="calculate the square root of {x}")]})
```

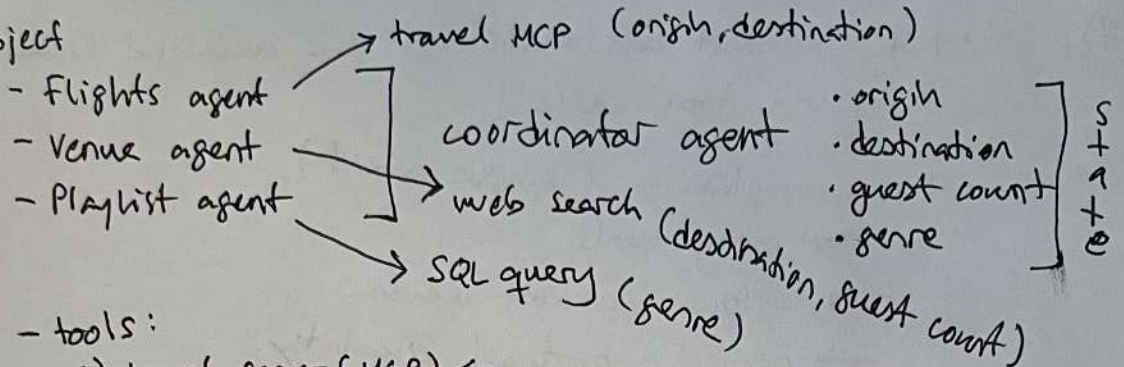
```
]})
```

```
return response["messages"][-1].content
```

```
response = main_agent.invoke({"messages": [HumanMessage(
    content="what is the square root of 456?")]})
```

⑧

Project



tools:

- 1) travel_server (MCP)
- 2) web_search
- 3) query_playlist_db

state

- origin
- destination
- guest_count
- genre

agents (specialized)

- 1) travel agent
- 2) venue agent
- 3) playlist agent
- +
- 4) coordinator main agent

tools: search_flights
(agents) search_venues
suggest_playlist
update_state
questions

Notes / observations:

- You are not allowed to ask any follow-up questions
- Access runtime state in tools: runtime.state["origin"]
- Folders for files
 - tools/
 - agents/
 - response_formats/
 - models / (?)
 - states /
 - contexts /
 - prompts /
 - middleware /

await coordinator.invoke({ "messages": [HumanMessage(content = "I'm from London and I'd like a wedding in Paris for 100 guests, jazz-genre")] })

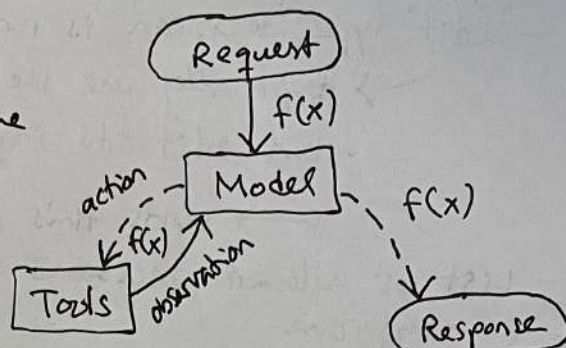
- AI Message: update_state tool call (parsed values)
- Tool Message: successfully updated state
- AI Message: search_flights, search_venues, suggest_playlist
- Tool Message: shortlist of flights
- Tool Message: wedding venues
- Tool Message: jazz playlist of 20 tracks w/ prices
- AI Message: final answer with all the details (wedding plan)

⑨ Module 3: Production-Ready Agent

- Middleware is a catch all term for functions that we can insert within these loops which allow us to control and customize our agent's execution.

- Long-term or long-running conversations cannot fit the model's context window

→ solution: middleware to summarize or delete/trim messages



- from langchain.agents.middleware import SummarizationMiddleware
 - model, trigger, keep (tokens) (messages) ← examples

→ Result: HumanMessage(... summary)
 HumanMessage(... ~~kept~~ kept message #1)
 AIMessage(... response final)

- @before_agent f(x)
 - @after_agent f(x)
 - @before_model f(x)
 - @after_model f(x)
- Run once, before or after each run
- Run multiple times, before or after each model call

→ Use cases:

- Remove all tool messages from the state → @before_agent
- uses RemoveMessage & filtering of messages

• Imports:

from langchain.agents.middleware import before_agent, after_agent, before_model, after_model

- Human-in-the-Loop (HITL)

- use cases:

- 1) approve sensitive actions (e.g. send email)
- 2) add missing context
- 3) debug the agent

- from langchain.agents.middleware import HumanInTheLoopMiddleware

- Interrupt on selected tool calls:

- False (default)

- True

→ { "interrupt": True, "messages": [...] } in response
 → final AIMessage(context="") w/ tool call

(10)

- thread_id needs to be included!
- from langgraph.types import Command
 - to resume with a decision → incl. message text (reason)
- "approve"/"reject" type decisions are straightforward
- "edit" type decision is more complex
 - typically use the same tool ~~name~~ but edit its input parameter value(s) (args)
 - sends this edited tool call
- List of allowed decisions:
 - approve
 - edit
 - reject
- Agents can dynamically adjust to user preferences and types
 - change prompts, tools, and models of the agent on the fly
 - solution: custom middleware
- Node-style middleware: added before ~~at~~ after the model or agent was invoked
- Wrap-style middleware: adjusts model request using a function. Change model, available tools, prompt,...
- Model request: $(F(x))$
 - model
 - prompt
 - state
 - tools
- from langchain.agents.middleware import dynamic_prompt, ModelRequest
- @dataclass
- class LanguageContext:
 - user_language: str = "English" ~~uses~~ ~~user~~ ~~prompt~~
- @dynamic_prompt → dynamic ~~prompt~~ ~~prompt~~
- def user_language_prompt(request: ModelRequest) → str:
 - """ generates system prompt based on user role """
 - user_language = request.runtime.context.user_language
 - base_prompt = "..."
 - if user_language != "English":
 - return f"{base_prompt} only respond in {user_language}."
 - else:
 - return base_prompt

⑪

```
agent = create_agent(  
    model = "gpt-5-nano",  
    context_schema = LanguageContext,  
    middleware = [user_language_prompt])  
agent.invoke({"messages": [...]}  
             context = LanguageContext(user_language="Irish"))
```

- from langchain.agents.middleware import wrap_model_call,
 from typing import Callable
 ModelRequest,
 ModelResponse
 @wrap_model_call
 def dynamic_tool_call(request: ModelRequest,
 handler: Callable[[ModelRequest],
 ModelResponse])
 → ModelResponse :
 """Dynamically call tools based on the runtime context"""
 user_role = request.runtime.context.user_role
 if user_role == "internal":
 pass // all tools
 else:
 tools = [web_search] // only this tool
 request = request.override(tools=tools)
 return handler(request)

```
agent = create_agent(  
    model = "gpt-5-nano",  
    tools = [web_search, sql_query],  
    middleware = [dynamic_tool_call],  
    context_schema = UserRole)  
agent.invoke({"messages": [...]}  
             context = {"user_role": "external"})
```

- from langchain.agents.middleware import wrap_model_call, ...
 @wrap_model_call
 def state_based_model(...) → ModelResponse :
 """selects model based on state conversation length"""
 ...
 request = request.override(model=model)
 return handler(request)

(12)

wrap-style middleware	node-style middleware	tool calls
ModelRequest	State + Runtime	Tool/Runtime
Adjust an agent's model, tools, prompt while running	Adjust an agent's state while running	Agent adjusts its own <u>state</u> or retrieves runtime context
e.g., giving an agent access to tools if the user is internal	e.g., Trimming messages from long conversations	e.g., Updating the state with the username when provided to the agent

- Email assistant (project)
 - authenticate user (to read emails)
 - approve emails to be sent (HTML)
- Agent Chat UI (Bonus)
 - convert @wrap_model_call functions to ~~async def~~
 async def...
 return await handler(request)
 - agentchat.vercel.app
Notebook to python file to localhost:2024
(hengsmith)
 - Agent Chat UI in the browser with UI widgets
 - hide tool calls
 - upload PDF or image
 - HTML interactive
 - Github repo: langchain-ai/agent-chat-ui
 - customize UI
 - interact with any LangGraph agent
via a chat interface UI
 - cd notebooks/module-3/agent-chat-ui
 - create .env
 - install dependencies (pnpm install)
 - run the app (pnpm dev) → http://localhost:3000